

빅데이터 분석 연습문제 풀이

② Q1. 다음 단어 리스트를 (1) One-Hot Encoding과 (2) StringIndexer 방식으로 벡터화하시오. (대소문자 무시)

```
['Grape', 'orange', 'melon', 'Orange', 'Peach', 'grape']
```

③ 상세 풀이 >

주어진 단어 리스트를 두 가지 다른 방식으로 벡터화합니다.

1. One-Hot Encoding 방식

One-Hot Encoding이란?

One-Hot Encoding은 범주형 데이터를 수치형 벡터로 변환하는 기법입니다. 단어 집합(Vocabulary)의 크기와 동일한 차원의 벡터를 만들고, 각 단어에 해당하는 인덱스의 값만 1로, 나머지 값은 모두 0으로 설정합니다. 이 방식은 단어 간의 순서나 유사성을 가정하지 않기 때문에 많은 머신러닝 모델의 입력으로 사용됩니다.

문제 풀이 적용:

1. 단어 소문자화 및 단어 집합(Vocabulary) 생성:

- 주어진 단어 리스트: ['Grape', 'orange', 'melon', 'Orange', 'Peach', 'grape']
- 모두 소문자로 변경: ['grape', 'orange', 'melon', 'orange', 'peach', 'grape']
- 고유 단어 집합 생성 (알파벳순 정렬): {grape, melon, orange, peach}
- 각 단어에 고유 인덱스 할당: grape: 0, melon: 1, orange: 2, peach: 3

2. One-Hot 벡터 생성:

단어 집합의 크기는 4이므로 4차원 벡터가 생성됩니다. 원본 리스트의 각 단어는 다음과 같이 변환됩니다.

- 'grape' → [1, 0, 0, 0]
- 'orange' → [0, 0, 1, 0]
- 'melon' → [0, 1, 0, 0]
- 'Orange' (orange) → [0, 0, 1, 0]
- 'Peach' (peach) → [0, 0, 0, 1]
- 'grape' → [1, 0, 0, 0]

최종 One-Hot Encoding 결과 (벡터의 리스트):

```
[
  [1, 0, 0, 0],
  [0, 0, 1, 0],
  [0, 1, 0, 0],
  [0, 0, 1, 0],
  [0, 0, 0, 1],
  [1, 0, 0, 0]
]
```

2. StringIndexer 방식

StringIndexer란?

StringIndexer는 Apache Spark와 같은 머신러닝 프레임워크에서 주로 사용되는 기법으로, 문자열로 된 범주형 데이터를 정수 인덱스로 변환합니다. 일반적으로 각 단어의 등장 빈도가 높은 순서대로 인덱스(0, 1, 2, ...)를 할당합니다.

문제 풀이 적용:

1. 단어 빈도수 계산:

- grape : 2번
- orange : 2번
- melon : 1번
- peach : 1번

2. 빈도 기반 인덱스 할당:

- 빈도수가 높은 순서대로 인덱스를 할당합니다. grape 와 orange 는 빈도가 2로 같으므로 알파벳순으로 정렬하여 grape 에 먼저 인덱스를 할당합니다.
- grape : 0
- orange : 1
- melon : 2
- peach : 3

3. 인덱스 벡터 생성:

원본 리스트 ['grape', 'orange', 'melon', 'orange', 'peach', 'grape'] 의 각 단어를 위에서 할당된 인덱스로 변환합니다.

최종 **StringIndexer** 결과 (인덱스의 리스트):

```
[0, 1, 2, 1, 3, 0]
```

빈도가 같은
알파벳순으로 정렬하

② Q2. 제조 회사는 전체 불량률 0.2%를 유지하고 있다. 품질 체크 부서를 기존 1개에서 5개로 늘린다. Bonferroni Correction 가정 시 각 부서의 허용 불량률은?

③ 상세 풀이 >

본페로니 교정(Bonferroni Correction)이란?

본페로니 교정은 다중 비교 문제(multiple comparisons problem)를 해결하기 위한 통계적 방법 중 하나입니다. 여러 가설을 동시에 검정할 때 전체 오류 확률(Family-wise error rate, FWER)이 증가하는 것을 막기 위해 사용됩니다.

예를 들어, 유의수준(p-value의 기준)을 0.05(5%)로 설정하고 가설 1개를 검정하면, 이 가설이 실제로 참인데도 우연히 기각될 확률은 5%입니다. 하지만 독립적인 가설 5개를 동시에 검정하면, 적어도 하나 이상의 가설이 우연히 잘못 기각될 확률은 다음과 같이 계산됩니다.

$$P(\text{적어도 하나가 잘못 기각될 확률}) = 1 - (1 - 0.05)^5 \approx 0.226$$

이는 약 22.6%까지 치솟게 됩니다.

본페로니 교정은 이 문제를 매우 간단하고 보수적인 방법으로 해결합니다. 개별 검정에 사용할 유의수준을 원래 유의수준(α)에서 검정하는 가설의 개수(n)로 나누는 것입니다.

- 수정된 유의수준 (α') = α/n

이렇게 하면 모든 검정을 통틀어 적어도 하나의 오류를 범할 확률(FWER)이 원래 유의수준 α 이하로 유지되는 것이 보장됩니다.

문제 풀이:

주어진 정보:

- 전체 허용 불량률 (α): $0.2\% = 0.002$
- 품질 검사 부서 수(테스트 횟수, n): 1개에서 5개로 증가. 즉, $n = 5$

계산:

각 부서는 독립적으로 불량률을 검사한다고 볼 수 있습니다. 따라서 5개 부서가 각각 불량률을 검사하는 것은 5개 가설을 동시에 검정하는 것과 같습니다. 이 상황에서 전체 허용 불량률 0.2%를 유지하려면 본페로니 교정을 적용해야 합니다.

각 부서에 적용할 새로운 허용 불량률(수정된 유의수준)은 다음과 같이 계산됩니다.

$$\alpha' = \frac{\text{전체 허용 불량률 } (\alpha)}{\text{부서 수 } (n)} = \frac{0.002}{5} = 0.0004$$

백분율로 변환하면 **0.04%** 입니다.

답:

본페로니 교정을 적용했을 때 각 부서의 허용 불량률은 **0.04%**가 되어야 합니다.

② Q3. 정규화(Normalization)의 필요성을 1가지 이상 설명하시오.

② 상세 풀이 >

정규화(Normalization)의 필요성

정규화는 데이터 전처리 과정에서 매우 중요한 단계입니다. 데이터셋의 여러 변수(feature)들의 값 범위(scale)가 서로 크게 다를 때, 이를 일정한 범위로 조정하는 역할을 합니다. 정규화가 필요한 주된 이유는 다음과 같습니다.

1. 머신러닝 모델 성능 향상

- **거리 기반 알고리즘:** K-최근접 이웃(KNN), 서포트 벡터 머신(SVM), 클러스터링과 같이 데이터 간의 거리를 기반으로 작동하는 알고리즘은 값의 범위에 매우 민감합니다. 예를 들어, 한 변수는 0~1 사이의 값을 갖고 다른 변수는 0~1,000,000 사이의 값을 갖는다면, 거리 계산은 거의 두 번째 변수에 의해서만 결정될 것입니다. 이는 모델이 다른 변수의 중요도를 제대로 학습하지 못하게 만듭니다. 정규화를 통해 모든 변수가 모델 학습에 동등하게 기여하도록 만들 수 있습니다.
- **경사 하강법(Gradient Descent) 기반 알고리즘:** 선형 회귀, 로지스틱 회귀, 신경망 등 경사 하강법을 사용하여 최적의 파라미터를 찾는 알고리즘에서도 정규화는 중요합니다. 변수들의 값 범위가 크게 다르면, 비용 함수(cost function)의 등고선이 한쪽으로 길게 찌그러진 타원 형태가 됩니다. 이 경우 최적점을 찾아가는 과정이 매우 비효율적이고 불안정해질 수 있습니다. 정규화를 통해 등고선을 원형에 가깝게 만들어주면, 경사 하강법이 더 빠르고 안정적으로 최적점에 수렴할 수 있습니다.

2. 피처별 스케일 차이로 인한 특정 피처 지배 방지

값의 범위가 큰 피처가 값의 범위가 작은 피처보다 모델 학습에 더 큰 영향(지배)을 미치는 것을 방지합니다. 정규화를 통해 모든 피처를 동일한 스케일로 조정하여, 특정 피처의 값 범위가 크다는 이유만으로 모델의 학습을 지배하거나 결과에 과도하게 영향을 미치지 않도록 합니다. 이를 통해 모델이 모든 피처의 중요도를 공정하게 반영하여 학습할 수 있게 됩니다.

대표적인 정규화 방법 (스케일러)

- **최소-최대 정규화 (Min-Max Scaler):**
모든 피처의 값을 0과 1 사이의 범위로 변환합니다. 데이터의 분포를 유지하면서 스케일을 조정할 때 유용합니다.

$$X_{\text{scaled}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

- **표준화 (StandardScaler):**
데이터의 분포를 평균이 0, 표준편차가 1인 표준 정규분포(가우시안 분포)로 변환합니다. 이상치(outlier)가 있는 데이터에 더 강건(robust)하며, 많은 머신러닝 알고리즘에서 기본적으로 선호되는(중점적으로 사용되는) 방법입니다.

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma}$$

- μ : 피처의 평균 (mean)

- σ : 피처의 표준편차 (standard deviation)

정규화의 효과 (그래프 해석)

경사 하강법 기반 알고리즘에서 정규화의 효과는 비용 함수(Cost Function)의 등고선(Contour Plot) 그래프를 통해 시각적으로 확인할 수 있습니다.

- **정규화 이전 (찌그러진 등고선)**: 피처 간의 스케일 차이가 크면, 등고선이 한쪽으로 길게 찌그러진 타원 형태가 됩니다. 이 경우 경사 하강법 알고리즘이 최적점을 향해 나아가는 경로가 매우 비효율적이고 지그재그로 움직여 수렴 속도가 느립니다.
- **정규화 이후 (원형 등고선)**: 정규화를 통해 피처들의 스케일을 맞추면, 등고선이 원형에 가까워집니다. 이것이 '좋은 데이터'의 시각적 형태이며, 이 경우 경사 하강법 알고리즘이 최적점을 향해 훨씬 빠르고 안정적으로 직선에 가깝게 수렴할 수 있습니다.

🔗 Q4. 버스 대기시간 데이터

```
wait_time=[2,5,12,18,18,25,33,35,1,7]
```

1. PMF 에서 $P(X = 18)$ 과 $P(X = 7)$ 을 계산하시오.
2. CDF 에서 $P(X \leq 20)$ 을 구하시오.

🔗 상세 풀이 >

주어진 데이터:

```
wait_time = [2, 5, 12, 18, 18, 25, 33, 35, 1, 7]
```

데이터 총 개수(n) = 10

데이터를 오름차순으로 정렬하면 다음과 같습니다.

```
[1, 2, 5, 7, 12, 18, 18, 25, 33, 35]
```

1) PMF(확률 질량 함수)에서 $P(X = 18)$ 과 $P(X = 7)$ 을 계산하시오.

PMF(Probability Mass Function, 확률 질량 함수)란?

PMF는 이산 확률 변수(discrete random variable)가 특정 값을 가질 확률을 나타내는 함수입니다. 어떤 변수 X 가 값 x 를 가질 확률을 $P(X = x)$ 로 표현합니다. 주어진 데이터 샘플에서 특정 값에 대한 PMF는 다음과 같이 계산할 수 있습니다.

$$P(X = x) = \frac{x \text{가 나타난 횟수}}{\text{전체 데이터 개수}}$$

문제 풀이:

- **$P(X = 18)$ 계산:**
 - `wait_time` 데이터에서 값 18은 **2번** 나타납니다.
 - 전체 데이터 개수는 **10개**입니다.
 - 따라서,

$$P(X = 18) = \frac{2}{10} = 0.2$$

- **$P(X = 7)$ 계산:**
 - `wait_time` 데이터에서 값 7은 **1번** 나타납니다.
 - 전체 데이터 개수는 **10개**입니다.
 - 따라서,

$$P(X = 7) = \frac{1}{10} = 0.1$$

2) CDF(누적 분포 함수)에서 $P(X \leq 20)$ 을 구하시오.

CDF(Cumulative Distribution Function, 누적 분포 함수)란?

CDF는 확률 변수 X 가 특정 값 x 보다 작거나 같을 확률을 나타내는 함수입니다. $P(X \leq x)$ 로 표현합니다. 즉, 가장 작은 값부터 x 까지의 모든 확률(PMF)을 더한 값입니다.

$$P(X \leq x) = \frac{\text{값이 } x \text{보다 작거나 같은 데이터의 개수}}{\text{전체 데이터 개수}}$$

변형된 문제 예시:

이 문제는 "내가 버스를 20분 기다리면 버스를 탈 확률은?"과 같이 실생활 문제로 변형되어 출제될 수 있습니다. 이는 $P(X \leq 20)$ 을 계산하는 것과 동일합니다.

Q5. 다음 RDD 연산 중 Transformation 만 모두 고르시오

함수명	반환형
<code>flatMap()</code>	<code>RDD[T] → RDD[U]</code>
<code>reduceByKey()</code>	<code>RDD[(K,V)] → RDD[(K,V)]</code>
<code>collect()</code>	<code>RDD[T] → Array[T]</code>
<code>take(5)</code>	<code>RDD[T] → Array[T]</code>
<code>distinct()</code>	<code>RDD[T] → RDD[T]</code>

상세 풀이 >

RDD 연산의 종류: 변환(Transformation) 대 액션(Action)

Apache Spark의 RDD(Resilient Distributed Dataset) 연산은 크게 두 가지로 나뉩니다.

1. 변환(Transformation):

- 기존 RDD를 기반으로 새로운 **RDD**를 생성하는 연산입니다.
- 예를 들어, 모든 요소에 2를 곱하거나 특정 조건을 만족하는 요소만 필터링하는 등의 작업을 수행합니다.
- 변환 연산은 "지연 실행(Lazy Evaluation)"됩니다. 즉, 코드가 호출되는 즉시 실행되는 것이 아니라 액션 연산이 호출될 때까지 실행 계획(DAG)에 기록만 됩니다.
- 반환 유형(Return Type)은 `RDD[...]` 형태입니다.

2. 액션(Action):

- RDD에 대한 계산을 수행하고 그 결과를 드라이버 프로그램으로 반환하거나 외부 저장소에 저장하는 연산입니다.
- 변환(Transformation) 연산과는 달리, 액션 연산은 지연 실행(Lazy Evaluation)으로 쌓여 있던 모든 변환들을 실제로 실행하도록 트리거(trigger)하는 역할을 합니다. 즉, 액션이 호출되는 시점에 비로소 실제 계산이 시작됩니다.
- 반환 유형은 RDD가 아닌 다른 값(예: `Array[T]`, `Long`, `Unit` 등)입니다.

문제 풀이:

문제에 주어진 RDD 연산 중에서 변환을 선택하는 것은 연산의 결과로 새로운 **RDD**가 반환되는 것을 찾는 것과 같습니다. 각 함수의 반환 유형을 기준으로 판단할 수 있습니다.

함수 이름	반환 유형	연산 종류	설명
<code>flatMap()</code>	<code>RDD[T] → RDD[U]</code>	변환	입력 RDD를 받아 새로운 RDD[U] 를 반환합니다.
<code>reduceByKey()</code>	<code>RDD[(K,V)] → RDD[(K,V)]</code>	변환	입력 RDD를 받아 새로운 RDD[(K,V)] 를 반환합니다.
<code>collect()</code>	<code>RDD[T] → Array[T]</code>	액션	RDD의 모든 요소를 드라이버의 배열(Array)로 반환합니다.

함수 이름	반환 유형	연산 종류	설명
<code>take(5)</code>	<code>RDD[T] → Array[T]</code>	액션	RDD의 상위 5개 요소를 드라이버의 배열(Array)로 반환합니다.
<code>distinct()</code>	<code>RDD[T] → RDD[T]</code>	변환	입력 RDD에서 중복을 제거한 새로운 RDD[T] 를 반환합니다.

답:

따라서 변환에 해당하는 연산은 `flatMap()`, `reduceByKey()`, `distinct()` 입니다.

③ Q6. 다음 데이터에서 결측정보를 확인하세요. 결측치가 **MCAR**, **MAR** 여부인지 확인하세요. **MNAR**에 대한 가능성이 있다고 할때, 그에 대한 예시를 적으시오.

ID	나이	근무경력(년)	급여	거주지
1	28	2	3200	서울
2	39	-	4500	부산
3	-	10	-	대전
4	45	20	7000	-
5	30	5	3800	서울

③ 상세 풀이 >

결측치(Missing Value)의 종류

데이터 분석에서 결측치는 피할 수 없는 문제입니다. 결측이 발생하는 메커니즘을 이해하는 것은 이를 적절히 처리하는 데 매우 중요합니다. 결측 메커니즘은 주로 세 가지로 분류됩니다.

1. MCAR(Missing Completely At Random, 완전 무작위 결측):

- 결측이 발생할 확률이 관측된 데이터나 관측되지 않은 데이터 모두와 아무런 관련이 없는 경우입니다. 즉, 말 그대로 '완전히 무작위로' 결측이 발생합니다.
- 예: 설문조사 응답자가 실수로 특정 문항을 빠뜨리고 제출한 경우.

2. MAR(Missing At Random, 무작위 결측):

- 결측이 발생할 확률이 관측된 다른 변수에는 의존하지만, 결측된 값 자체와는 관련이 없는 경우입니다.
- 예: 남성보다 여성이 자신의 몸무게를 더 자주 응답하지 않는 경향이 있다면, '몸무게' 변수의 결측은 '성별'이라는 관측된 변수와 관련이 있습니다. 하지만 동일한 성별 내에서는 몸무게 값 자체(즉, 몸무게가 많이 나가서 숨기는 것)와는 관련이 없다고 가정합니다.

3. MNAR(Missing Not At Random, 비무작위 결측):

- 결측이 발생할 확률이 결측된 값 자체와 관련이 있는 경우입니다. 가장 처리하기 어려운 유형입니다.
- 예: 소득이 매우 높은 사람들이나 매우 낮은 사람들이 자신의 소득을 공개하기를 꺼려 소득 정보가 누락되는 경우. '소득' 변수의 결측이 '소득' 값 자체와 직접적인 관련이 있습니다.

문제 풀이:

1. 결측 정보 확인:

- ID 2: 근무경력(년) 결측
- ID 3: 나이, 급여 결측
- ID 4: 거주지 결측

2. MCAR, MAR, MNAR 시나리오 서술:

- **MCAR(완전 무작위 결측) 가능성:**

ID 3의 나이 값이 특별한 이유 없이, 예를 들어 단순 입력 실수나 시스템 오류로 '그냥' 누락된 경우입니다. 이 경우 결측은 다른 어떤 변수나 나이 값 자체와도 관련이 없습니다.

- **MAR(무작위 결측) 가능성:**

근무경력 결측이 관측된 다른 변수인 급여나 거주지 와 관련이 있는 시나리오입니다.

- 예시 1: 급여가 4000~6000 사이인 사람들이 근무경력 을 잘 입력하지 않는 경향이 있다.
 - 예시 2: 거주지 가 부산이거나 남쪽 도시인 경우, 근무경력 정보가 누락될 가능성이 높다.
- 이처럼 결측이 다른 '관측된' 변수로 설명될 수 있다면 MAR 입니다.

- **MNAR(비무작위 결측) 가능성:**

결측된 값 자체가 결측의 원인이 되는 경우입니다. 이 문제의 핵심은 새로운 가상의 컬럼을 만들어서라도 MNAR의 가능성을 논리적으로 설명할 수 있어야 한다는 점입니다.

- 예시: 급여 결측(ID 3)이 MNAR인 시나리오

데이터셋에 없는 세금이라는 가상의 컬럼이 있다고 가정해 봅시다. 급여가 높을수록 세금도 높아집니다. 만약 급여가 매우 높은 사람들이 높은 세금 부담 때문에 자신의 급여 공개를 꺼려 정보를 입력하지 않았다면, 이는 급여라는 값 자체가 높기 때문에 결측이 발생한 것입니다. 이처럼 결측된 값이 다른 변수(세금)를 통해 자기 자신과 직접적인 관련을 맺으면 MNAR에 해당합니다.

② Q7. Spark 에서 하나의 Job 이 실행되는 과정에서 나타나는 Stage-Task 구조를 계층(Hierarchy) 관점에서 설명하시오.

③ 상세 풀이 >

Spark 실행 계층 구조: Job, Stage, Task

Spark에서 하나의 코드를 실행하면, Spark는 이를 내부적으로 Job, Stage, Task라는 계층적인 단위로 나누어 분산 처리합니다. 이 구조는 Spark의 핵심인 DAG(Directed Acyclic Graph) 스케줄러에 의해 관리됩니다.

계층 구조:

```
[ Spark Application ]
  |
  [ Job ]
    |
    [ Stage 1 ]---[ Stage 2 ]---...
      |           |
      [ Task 1-1 ] [ Task 2-1 ]
      [ Task 1-2 ] [ Task 2-2 ]
      ...         ...
```

1. Application (애플리케이션):

- 가장 상위 개념으로, 사용자가 제출한 하나의 Spark 프로그램 전체를 의미합니다.
- 하나의 Spark Application은 하나 이상의 작업으로 구성됩니다.
- 예: spark-submit 으로 실행한 .py 또는 .jar 파일 하나.

2. Job (작업):

- Spark Application 내에서 하나의 액션 연산이 호출될 때마다 하나의 작업이 생성되고 실행됩니다. (즉, 액션을 통해 실행이 됩니다.)
- 액션 연산은 RDD에 대한 실제 계산을 촉발하고 결과를 반환(예: count(), collect())하거나 저장(예: saveAsTextFile())하는 역할을 합니다.
- 하나의 작업은 여러 단계로 나뉘어 실행됩니다.

3. Stage (스테이지):

- 작업(Job)은 여러 개의 스테이지(Stage)로 구성됩니다. 스테이지는 DAG(Directed Acyclic Graph)의 중간 계층을 이룹니다.
- 하나의 작업은 셔플(Shuffle)이 발생하는 지점을 경계로 여러 단계로 분리됩니다. (즉, 셔플을 경계로 분할하고 파티션하는 것이 목적입니다.)
- 셔플이란 `reduceByKey`, `groupByKey`, `join` 과 같이 네트워크를 통해 데이터를 재분배하는 과정을 의미합니다. 즉, 스테이지는 셔플 없이 연속적으로 수행할 수 있는 변환들의 묶음입니다.
- 각 스테이지는 또한 여러 개의 태스크(Task)로 이루어집니다.
- 같은 스테이지 내의 태스크들은 데이터 이동 없이 파이프라이닝(pipelining) 방식으로 동시에 실행될 수 있어 매우 효율적입니다.

4. Task (태스크):

- 가장 작은 실행 단위로, 하나의 스테이지는 여러 태스크로 구성됩니다.
- 태스크는 하나의 파티션(Partition)에 대해 하나의 CPU 코어(Executor의 core)에서 실행되며, 파티션별로 병렬로 실행하는 것이 목적입니다.
- 각 태스크는 Executor의 CPU 코어 하나에 할당되어 작업을 수행하며, 동시에 여러 태스크가 다른 코어에서 병렬로 실행될 수 있습니다.
- 예를 들어, 어떤 스테이지가 100개의 파티션을 가진 RDD를 처리한다면, 해당 스테이지는 100개의 태스크를 생성하여 각 파티션에 대해 동일한 연산을 병렬로 수행합니다.

② Q8. 다음 Spark 코드의 line 번호 중 cache 가 영향을 미치는 구간(연산 line)을 모두 쓰시오.

```
1 val data = sc.textFile("hdfs://...")
2 val filtered = data.filter(_.contains("INFO"))
3 val parsed = filtered.map(_.split(","))
4 parsed.cache()
5 parsed.filter(_(2).toInt > 50).count()
```

② 상세 풀이 >

Spark의 `cache()` 연산과 영향 범위

Spark에서 `cache()` 또는 `persist()` 는 RDD를 메모리나 디스크에 저장하여 재사용할 때의 성능을 향상시키는 매우 중요한 변환(Transformation) 연산입니다.

`cache()` 의 작동 원리:

- `cache()` 는 지연 실행(Lazy Evaluation)되는 변환입니다. 즉, `parsed.cache()` 줄이 호출되는 순간에는 아무 일도 일어나지 않습니다.
- `cache()` 가 적용된 RDD(`parsed`)에 대해 최초의 액션(Action) 연산(여기서는 `count()`)이 실행될 때, Spark는 이 RDD를 계산하여 결과를 반환함과 동시에 `parsed` RDD의 내용이 각 Executor의 메모리에 캐시(저장)됩니다.
- 이후에 동일한 `parsed` RDD를 사용하는 다른 액션 연산이 호출되면, Spark는 원본 데이터 소스(HDFS)로부터 복잡한 계산(textFile -> filter -> map)을 다시 수행하는 대신, 메모리에 캐시된 RDD를 즉시 읽어서 사용합니다. 이로 인해 연산 속도가 매우 빨라집니다.

문제 풀이:

주어진 Spark 코드는 다음과 같습니다.

```
1 val data = sc.textFile("hdfs://...")
2 val filtered = data.filter(_.contains("INFO"))
3 val parsed = filtered.map(_.split(","))
```



```
4 parsed.cache()
5 parsed.filter(_(2).toInt > 50).count()
```

cache() 가 영향을 미치는 구간(연산 줄):

cache() 는 parsed RDD와 그 이전의 모든 RDD 생성 과정에 영향을 미칩니다. 즉, parsed RDD를 계산하는 데 필요한 모든 연산을 "기억"하고 그 결과를 저장합니다.

- 5행의 count() 액션이 처음 실행될 때, parsed RDD가 없으므로 Spark는 DAG(실행 계획)에 따라 parsed 를 계산합니다.
- parsed 를 계산하려면 부모 RDD인 filtered 가 필요하고, filtered 를 계산하려면 원본 RDD인 data 가 필요합니다.
- 따라서 다음 연산이 순차적으로 실행됩니다.
 - 1행: sc.textFile(...) - data RDD 생성 (Transformation)
 - 2행: data.filter(...) - filtered RDD 생성 (Transformation)
 - 3행: filtered.map(...) - parsed RDD 생성 (Transformation)
- 4행의 cache() 명령에 따라 이렇게 생성된 parsed RDD의 내용이 각 Executor의 메모리에 저장됩니다.
- 그런 다음 5행의 나머지 연산(filter(...).count())이 수행됩니다.

만약 이 코드 뒤에 parsed.filter(_.contains("ERROR")).count() 와 같이 parsed RDD를 재사용하는 코드가 있는 경우, 1, 2, 3행을 다시 실행하지 않고 메모리에 저장된 parsed RDD를 즉시 사용합니다.

답:

문제에서 묻는 "cache가 영향을 미치는 구간"은 캐시를 생성하기 위해 실행되는 모든 연산 라인을 의미합니다. 따라서 답은 1, 2, 3번입니다.

중요: 캐시의 혜택을 보는 구간은 다릅니다. 캐시는 line 4 이후 parsed 변수를 사용하는 모든 연산의 성능을 향상시킵니다. 예를 들어, line 5의 count() 연산도 캐시를 생성함과 동시에 캐시된 데이터를 사용하며, 이후에 parsed 를 재사용하는 모든 연산은 캐시의 혜택을 직접적으로 받게 됩니다.

㉠ Q9. 4 일 동안의 지각 학생 수에 대한 관측값과 기대값이 다음과 같을 때, 유의수준 5%에서 귀무가설("요일 간 차이가 없다")의 기각 여부를 판단하시오.

요일	관측값	기대값
월	5	7
화	10	7
수	8	7
목	5	7

(자유도 df = 3, 표 제공됨)

df	0.995	0.99	0.975	0.95	0.90	0.10	0.05	0.025	0.01	0.005
1	---	---	0.001	0.004	0.016	2.706	3.841	5.024	6.635	7.879
2	0.010	0.020	0.051	0.103	0.211	4.605	5.991	7.378	9.210	10.597
3	0.072	0.115	0.216	0.352	0.584	6.251	7.815	9.348	11.345	12.838
4	0.207	0.297	0.484	0.711	1.064	7.779	9.488	11.143	13.277	14.860
5	0.412	0.554	0.831	1.145	1.610	9.236	11.070	12.833	15.086	16.750
6	0.676	0.872	1.237	1.635	2.204	10.645	12.592	14.449	16.812	18.548
7	0.989	1.239	1.690	2.167	2.833	12.017	14.067	16.013	18.475	20.278
8	1.344	1.646	2.180	2.733	3.490	13.362	15.507	17.535	20.090	21.955
9	1.735	2.088	2.700	3.325	4.168	14.684	16.919	19.023	21.666	23.589
10	2.156	2.558	3.247	3.940	4.865	15.987	18.307	20.483	23.209	25.188

카이제곱 검정을 이용한 가설 검정

카이제곱 검정은 관측된 빈도가 기대 빈도와 통계적으로 유의미하게 다른지 판단하는 데 사용되는 가설 검정 방법입니다. 주로 범주형 데이터 분석에 사용됩니다.

가설 설정:

- 귀무가설(H_0): 관측값과 기대값 사이에 차이가 없다. (즉, 요일별 지각 학생 수에 차이가 없다.)
- 대립가설(H_1): 관측값과 기대값 사이에 차이가 있다. (즉, 요일별 지각 학생 수에 차이가 있다.)

카이제곱 통계량 계산:

카이제곱 통계량(χ^2)은 각 범주에 대해 (관측값 - 기대값)² / 기대값을 모두 더한 값입니다.

$$\chi^2 = \frac{(O - E)^2}{E}$$

O : 관측 빈도 (Observed)

E : 기대 빈도 (Expected)

문제 풀이:

1. 카이제곱 통계량 계산:

요일	관측값 (O)	기대값 (E)	(O - E)	(O - E) ²	(O - E) ² / E
월	5	7	-2	4	4 / 7 ≈ 0.571
화	10	7	3	9	9 / 7 ≈ 1.286
수	8	7	1	1	1 / 7 ≈ 0.143
목	5	7	-2	4	4 / 7 ≈ 0.571
합계	28	28			$\chi^2 \approx 2.571$

$$\chi^2 = \frac{(5-7)^2}{7} + \frac{(10-7)^2}{7} + \frac{(8-7)^2}{7} + \frac{(5-7)^2}{7} = \frac{4+9+1+4}{7} = \frac{18}{7} \approx 2.571$$

2. 기각 여부 판단:

계산된 카이제곱 통계량을 기각역(critical value)과 비교하여 귀무가설의 기각 여부를 판단합니다. 기각역은 유의수준(α)과 자유도(df)에 따라 결정됩니다.

- 유의수준 (α): 0.05 (5%) (문제에 따라 '시그니피칸스(Significance) 레벨'로 표현될 수도 있습니다.)
- 자유도 (df): 범주의 수(k) - 1

$$df = 4 - 1 = 3$$

(참고: $df = (\text{행의 수} - 1) \times (\text{열의 수} - 1)$ 은 독립성 검정(Test of Independence)을 위한 카이제곱 검정에서 사용됩니다. 이 문제는 관측 빈도가 특정 기대 분포(여기서는 균일 분포)를 따르는지 검정하는 적합도 검정(Goodness-of-Fit Test)이므로, 자유도는 범주의 수(k) - 1로 계산됩니다.)

문제에서 "표 제공됨"이라고 언급했으므로, 우리는 카이제곱 분포표에서 유의수준 0.05, 자유도 3에 해당하는 임계값을 찾아야 합니다.

일반적인 카이제곱 분포표에서, $df=3$, $\alpha=0.05$ 일 때의 임계값은 7.815입니다.

3. 결론:

- 계산된 카이제곱 통계량($\chi^2_{\text{calc}} \approx 2.571$)은 임계값($\chi^2_{\text{crit}} = 7.815$)보다 작습니다.
 $2.571 < 7.815$
- 통계량이 임계값보다 작으므로, 귀무가설을 기각할 충분한 근거가 없습니다.

답:

계산된 카이제곱 통계량(약 2.571)이 유의수준 5%, 자유도 3에서의 임계값 7.815보다 작으므로, 귀무가설("요일 간 차이가 없다")을 기각할 수 없습니다. 즉, 요일별 지각 학생 수의 차이는 통계적으로 유의미하다고 볼 수 없습니다.

② Q10. 다음은 임의의 3×3 행렬 A 에 대한 SVD 결과이다. SVD의 특이값은 다음과 같다.

$$\sigma_1 = 7, \sigma_2 = 3, \sigma_3 = 1$$

- 차원 축소로, 상위 $k=2$ 개만 취하는 경우, 전체 에너지 대비 오차의 비율(오차율, Error Ratio)을 계산하시오.

③ 상세 풀이 >

SVD(Singular Value Decomposition, 특이값 분해)와 에너지

SVD는 행렬 A 를 다음과 같이 세 개의 행렬 곱으로 분해하는 기법입니다.

$$A = U \Sigma V^T$$

- U, V : 직교 행렬(Orthogonal Matrices)
- Σ : 대각 행렬(Diagonal Matrix)

여기서 특이값(Singular Value)은 행렬 Σ 의 대각 성분($\sigma_1, \sigma_2, \sigma_3, \dots$)을 의미하며, 각 특이값은 데이터의 '에너지' 또는 '중요도'를 나타냅니다.

SVD를 이용한 차원 축소는 중요한 정보(에너지)를 대부분 가진 상위 k 개의 특이값만 남기고 나머지는 버리는 방식입니다. 이 과정에서 버려진 특이값들이 오차(Error)를 구성합니다.

문제 풀이:

1. 주어진 정보 해석

- 행렬 A 는 SVD를 통해 $A = U \Sigma V^T$ 로 분해됩니다.
- 문제에 주어진 특이값 $\sigma_1 = 7, \sigma_2 = 3, \sigma_3 = 1$ 은 행렬 Σ 의 주대각 성분입니다.

$$\Sigma = \begin{pmatrix} 7 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

2. 전체 에너지 계산

전체 에너지는 모든 특이값의 제곱의 합으로 계산됩니다.

$$\text{전체 에너지} = \sigma_1^2 + \sigma_2^2 + \sigma_3^2 = 7^2 + 3^2 + 1^2 = 49 + 9 + 1 = 59$$

3. 오차 계산 ($k=2$ 차원 축소)

- 상위 $k=2$ 개만 취한다는 것은, 가장 큰 특이값 2개($\sigma_1 = 7, \sigma_2 = 3$)만 남기고 나머지는 버리는 것을 의미합니다.
- 따라서 손실되는 정보, 즉 오차는 버려진 특이값($\sigma_3 = 1$)으로부터 계산됩니다.
- 손실된 에너지(오차)는 버려진 특이값의 제곱의 합입니다.

$$\text{손실된 에너지} = \sigma_3^2 = 1^2 = 1$$

4. 오차율 계산

오차율은 전체 에너지 대비 손실된 에너지의 비율입니다.

$$\text{오차율 (Error Ratio)} = \frac{\text{손실된 에너지}}{\text{전체 에너지}} = \frac{1}{59}$$

답:

상위 2개 특이값만 사용하는 경우, 전체 에너지 대비 오차 비율(오차율)은 $\frac{1}{59}$ 입니다. (약 1.69%)

③ Q11. Bloom Filter 정보가 다음과 같을 때, false positive 가 발생할 수 있는 원소를 찾으시오.

```
bit array size = 12
hash functions:
```

```
h1(x) = x % 12
h2(x) = (x // 10 + x % 10) % 12
inserted items: [15, 27, 39]
test items: [11, 24, 36, 51]
```

🔗 상세 풀이 >

블룸 필터(Bloom Filter)와 거짓 양성(False Positive)

블룸 필터는 어떤 원소가 집합에 속하는지 여부를 **확률적으로** 검사하는 데이터 구조입니다. 매우 적은 메모리를 사용하여 빠른 검사가 가능하지만, 다음과 같은 특징이 있습니다.

- **거짓 양성(False Positive) 가능성:** 집합에 없는 원소를 '있다'고 잘못 판단할 수 있습니다.
- **거짓 음성(False Negative) 없음:** 집합에 있는 원소를 '없다'고 판단하는 경우는 절대 없습니다.

문제 풀이 방법

블룸 필터 문제는 다음 두 단계로 해결할 수 있습니다.

1. **삽입 단계:** `inserted items` 의 각 원소를 해시 함수에 통과시켜 블룸 필터(비트 배열)의 해당 위치를 1로 변경합니다.
2. **검사 단계:** `test items` 의 각 원소를 해시 함수에 통과시켜, 계산된 위치의 비트들이 **모두 1인지** 확인합니다. 만약 모두 1이라면 '있다(found)'고 판단하고, 하나라도 0이면 '없다(not found)'고 판단합니다.

만약 집합에 없는 원소가 '있다'고 판단되면, 그것이 **거짓 양성(False Positive)**입니다.

문제 풀이 적용

1. 삽입 단계: 블룸 필터 구성

- 초기 상태: 크기 12의 비트 배열을 0으로 초기화합니다.
`[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]`
- 원소 15 삽입:
 - $h_1(15) = 15 \% 12 = 3$
 - $h_2(15) = (15 // 10 + 15 \% 10) \% 12 = (1 + 5) \% 12 = 6$
 - 3번, 6번 인덱스를 1로 변경: `[0, 0, 0, **1**, 0, 0, **1**, 0, 0, 0, 0, 0]`
- 원소 27 삽입:
 - $h_1(27) = 27 \% 12 = 3$
 - $h_2(27) = (27 // 10 + 27 \% 10) \% 12 = (2 + 7) \% 12 = 9$
 - 3번, 9번 인덱스를 1로 변경 (3번은 이미 1): `[0, 0, 0, **1**, 0, 0, **1**, 0, 0, **1**, 0, 0]`
- 원소 39 삽입:
 - $h_1(39) = 39 \% 12 = 3$
 - $h_2(39) = (39 // 10 + 39 \% 10) \% 12 = (3 + 9) \% 12 = 0$
 - 0번, 3번 인덱스를 1로 변경 (3번은 이미 1): `[**1**, 0, 0, **1**, 0, 0, **1**, 0, 0, **1**, 0, 0]`
- 최종 블룸 필터: `[1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0]`

2. 검사 단계: 거짓 양성 찾기

(참고: `test items` 가 `inserted items` 에 원래 포함되어 있고 블룸 필터가 **있음** 으로 판단한다면, 이는 정상적인 판단이며 참 양성(True Positive)입니다. 거짓 양성은 삽입되지 않은 원소를 **있음** 으로 잘못 판단하는 경우에만 해당합니다.)

- 원소 11 검사:

- $h_1(11) = 11$, $h_2(11) = (1+1)\%12 = 2$
- 비트배열[11] = 0, 비트배열[2] = 0. → '없음' (정상)
- 원소 24 검사:
 - $h_1(24) = 0$, $h_2(24) = (2+4)\%12 = 6$
 - 비트배열[0] = 1, 비트배열[6] = 1. → '있음' (거짓 양성!)
- 원소 36 검사:
 - $h_1(36) = 0$, $h_2(36) = (3+6)\%12 = 9$
 - 비트배열[0] = 1, 비트배열[9] = 1. → '있음' (거짓 양성!)
- 원소 51 검사:
 - $h_1(51) = 3$, $h_2(51) = (5+1)\%12 = 6$
 - 비트배열[3] = 1, 비트배열[6] = 1. → '있음' (거짓 양성!)

답:

inserted items 에 없는 원소 24, 36, 51 은 다른 원소들이 0번, 3번, 6번, 9번 비트를 1로 만드는 과정에서 우연히 '있음'으로 판단되었습니다.
따라서 거짓 양성이 발생하는 원소는 **24, 36, 51** 입니다.

② Q12. 다음 데이터는 **Flajolet-Martin** 알고리즘에서 해시 결과이다. 가장 오른쪽 1이 처음 등장하는 위치를 기반으로 고유 원소 수를 추정하시오.

값	해시값 (8bit)
A	01010000
B	00010010
C	00100000
D	10000001

③ 상세 풀이 >

Flajolet-Martin 알고리즘을 이용한 고유 원소 수 추정

Flajolet-Martin 알고리즘은 스트리밍 데이터에서 고유한 원소의 개수(카디널리티)를 근사적으로 추정하는 데 사용되는 확률적 알고리즘입니다. 매우 적은 메모리를 사용하여 대용량 데이터의 고유 원소 수를 빠르게 추정할 수 있습니다.

알고리즘 원리:

1. 해싱: 스트림의 각 원소를 비트 문자열로 변환하는 해시 함수 $h(x)$ 를 사용합니다. 이 해시 함수는 결과값이 비트 전체에 고르게 분포되어야 합니다.
2. $R(x)$ 계산: 각 해시값 $h(x)$ 에 대해, 가장 오른쪽(최하위 비트)부터 시작해서 처음으로 1이 나타나는 위치를 $R(x)$ 로 정의합니다. 예를 들어, 해시값이 **...1000** 이면 $R(x)=3$ 입니다. (0부터 시작하면)
3. 최댓값 유지: 스트림을 읽으면서 계산된 모든 $R(x)$ 값들 중 최댓값 R_{max} 를 계속 유지합니다.
4. 추정: 스트림의 모든 원소를 처리한 후, 고유 원소의 개수를 $2^{R_{max}}$ 로 추정합니다.

직관적인 이해:

고유한 원소가 N 개 있다면, $R(x)=k$ 인 해시값을 가질 확률은 $1/2^{(k+1)}$ 입니다. 즉, $R(x)$ 가 크려면 해시값 뒤에 0이 많이 붙어야 하는데, 이는 매우 드문 일입니다. 만약 우리가 N 개의 다른 원소를 해싱했을 때, R_{max} 가 k 라면, 이는 2^k 정도의 원소를 시도했을 때 한 번 나올 법한 희귀한 패턴이 나타났다는 것을 의미합니다. 따라서 고유 원소의 개수를 2^k (여기서는 $2^{R_{max}}$)라고 추정하는 것입니다.

문제 풀이:

주어진 정보:

문제에서는 "가장 오른쪽 1이 처음 등장하는 위치"를 기반으로 추정하라고 되어 있습니다. 이는 $R(x)$ 를 계산하는 것과 같습니다. (위치의 시작을 0으로 가정합니다.)

값	해시값 (8bit)	가장 오른쪽 1의 위치 (R)
A	01010000	4
B	00010010	1
C	00100000	5
D	10000001	0

1. 각 해시값에 대해 R값 계산:

- **A (01010000)**: 오른쪽에서 5번째 비트가 처음 나오는 1입니다. (위치를 0부터 세면 $R=4$)
- **B (00010010)**: 오른쪽에서 2번째 비트가 처음 나오는 1입니다. ($R=1$)
- **C (00100000)**: 오른쪽에서 6번째 비트가 처음 나오는 1입니다. ($R=5$)
- **D (10000001)**: 오른쪽에서 1번째 비트가 처음 나오는 1입니다. ($R=0$)

2. R의 최댓값($R_{\max}$) 찾기:

계산된 R 값들 {4, 1, 5, 0} 중에서 최댓값은 5입니다.

- $R_{\max} = 5$

3. 고유 원소 수 추정:

고유 원소의 수는 $2^{R_{\max}}$ 로 추정합니다.

- 추정된 고유 원소 수 = $2^5 = 32$

답:

Flajolet-Martin 알고리즘에 따라 추정된 고유 원소의 수는 32입니다.

참고: 실제 Flajolet-Martin 알고리즘은 정확도를 높이기 위해 여러 개의 해시 함수를 사용하고 그 결과의 평균(또는 중앙값)을 사용하는 등 더 복잡한 과정을 거치지만, 기본 원리는 위와 같습니다.

㉠ **Q13. Spearman Correlation** 계산 결과, 국어 성적과 영어 성적 간의 순위 상관계수 $\rho = 0.92$ 가 도출되었다. 이 결과의 의미를 서술하시오.

㉡ 상세 풀이 >

스피어만 순위 상관계수(Spearman Correlation)의 의미

상관계수는 두 변수 간의 관계가 얼마나 강한지를 나타내는 -1에서 1 사이의 값입니다. 스피어만 순위 상관계수(ρ 또는 r_s)는 일반적인 피어슨 상관계수(Pearson Correlation)와 중요한 차이점이 있습니다.

- 피어슨 상관계수: 두 변수 간의 선형(linear) 관계를 측정합니다. 변수의 실제 값(value)을 사용하여 계산합니다.
- 스피어만 순위 상관계수: 두 변수 간의 단조(monotonic) 관계를 측정합니다. 변수의 실제 값 대신, 값의 순위(rank)를 사용하여 계산합니다.

단조 관계(Monotonic Relationship)란?

한 변수가 증가할 때 다른 변수도 항상 같이 증가하거나(단조 증가), 항상 같이 감소하는(단조 감소) 관계를 의미합니다. 이 관계가 반드시 직선일 필요는 없습니다. 아래와 같은 곡선 관계도 단조 관계에 포함됩니다.

문제 풀이:

주어진 정보:

"국어 성적과 영어 성적 간의 순위 상관계수 $\rho = 0.92$ 가 도출되었다."

이 결과의 의미:

1. 매우 강한 양의 단조 관계(Monotonic Relationship):

- 상관계수 값 0.92는 **1에 매우 가깝습니다**. 이는 두 변수 사이에 매우 강한 양의 단조 관계가 존재함을 의미합니다.
- 즉, "국어 성적의 등수"가 높아지면 "영어 성적의 등수"도 함께 높아지는 경향이 매우 뚜렷하다는 뜻입니다.

2. 순위 기반의 관계:

- 이것은 "국어 점수"와 "영어 점수"의 관계가 아니라, "**국어 등수**"와 "**영어 등수**"의 관계를 의미합니다.
- 즉, 국어 성적 등수가 높은 학생은 영어 성적 등수도 높을 경향이 매우 강하다는 것을 뜻합니다. 반대로 국어 등수가 낮은 학생은 영어 등수도 낮을 가능성이 큼니다.

3. 선형 관계가 아닐 수 있음:

- 두 학생이 국어 점수에서 10점 차이가 날 때, 영어 점수에서도 항상 비슷한 점수 차이를 보인다는 의미는 아닙니다.
- 예를 들어, 국어 1등이 100점, 2등이 80점이고, 영어 1등이 100점, 2등이 99점일 수 있습니다. 점수 차이는 다르지만 '순위'는 1등, 2등으로 동일하게 유지됩니다. 스피어만 상관계수는 바로 이 순위의 일치성을 측정합니다.

상관계수 값에 따른 해석:

• 계수가 1에 가까울 경우 (예: 0.92):

두 변수의 순위가 거의 일치하며, 매우 강한 양의 단조 관계를 가집니다. 한 변수의 순위가 오르면 다른 변수의 순위도 거의 항상 오릅니다.

• 계수가 -1에 가까울 경우 (예: -0.9):

두 변수의 순위가 거의 반대로 나타나며, 매우 강한 음의 단조 관계를 가집니다. 한 변수의 순위가 오르면 다른 변수의 순위는 거의 항상 내려갑니다. (예: "수학 등수"가 오를수록 "게임 시간 등수"는 내려가는 관계)

• 계수가 0에 가까울 경우 (예: 0.1):

두 변수의 순위 간에는 뚜렷한 단조 관계가 거의 없습니다. 즉, 한 변수의 순위가 오르거나 내리는 것이 다른 변수의 순위에 거의 영향을 주지 않습니다.

결론:

국어 성적과 영어 성적 간의 스피어만 순위 상관계수가 0.92라는 것은, 두 과목의 상대적인 순위가 매우 유사하다는 것을 의미합니다. 국어를 잘하는 학생이 영어도 잘하고, 국어를 못하는 학생이 영어도 못하는 매우 강한 단조(monotonic) 관계가 나타난다고 해석할 수 있습니다.